Módulo LLM Local em Sistema Web para Geração de Relatórios Textuais via *Prompt*

Luiz Batista Cardoso, Alexandre Zamberlan
Curso de Ciência da Computação
UFN - Universidade Franciscana
Santa Maria - RS
luiz.bcardoso@ufn.edu.br, alexz@ufn.edu.br

***Resumo*—Este trabalho aborda a necessidade de aprimorar os sistemas de informação do Laboratório de Práticas da Computação da Universidade Franciscana (UFN), focando na interação humano-computador. Esses sistemas careciam de funcionalidades avançadas para geração automatizada de relatórios, exigindo consultas manuais complexas em SQL. Dessa forma, foi proposto desenvolver um módulo de interação textual, integrando Processamento de Linguagem Natural (PLN) e modelos de linguagem (LLMs) para converter consultas em perguntas para SQL, facilitando a geração de relatórios. Utilizaram-se as bibliotecas langchain, ollama, xmlrpc, o modelo Llama 3.1 8B e o framework LangChain para a implementação da Recuperação Aumentada (RAG). Tendo resultados consistentes e precisos para perguntas curtas e bem estruturadas quando comparadas com consultas manuais.**

***Palavras-chave*: Integração de sistemas; Computação em Nuvem; RPC; Inteligência Artificial; Python.**

I. Introdução

Os sistemas de informação projetados, desenvolvidos e disponibilizados no Laboratório de Práticas da Computação da Universidade Franciscana (UFN) são aplicados em diversas áreas. No entanto, tais sistemas, até o presente momento, apresentavam carência de funcionalidades avançadas de interação humano-computador, especialmente no que diz respeito a interfaces de *prompts* para consulta de dados em linguagem natural. Assim, projetar e integrar um módulo que permita a geração de relatórios detalhados com base nas informações armazenadas nos bancos de dados desses sistemas torna-se uma necessidade premente e relevante.

Até o segundo semestre de 2024, sistemas Web como o SIGEP-COMIC [1] não ofereciam módulos integrados para a geração automatizada de relatórios. Quando os coordenadores desses sistemas precisam de informações específicas, enfrentam um processo quase manual, recorrendo a filtros predefinidos e realizando consultas limitadas. Para melhorar essa situação, o módulo permite que os usuários possam formular suas solicitações de maneira descritiva e em português, facilitando a obtenção de relatórios personalizados e mais detalhados com maior eficiência e menor esforço manual. Para converter consultas em linguagem natural para SQL, é possível utilizar ferramentas de Processamento de Linguagem Natural (PLN), como *Meta Llama*. Essas ferramentas empregam Inteligência Artificial Generativa para realizar a conversão de texto para SQL. No contexto deste trabalho, um *Large Language Model* (LLM) desempenha papel crucial, facilitando a interpretação da linguagem natural e a geração de consultas SQL, entre outras funções.

O objetivo geral desta pesquisa foi desenvolver e integrar um módulo de interação textual em linguagem natural, via *prompt*, para a geração de relatórios em um sistema Web construído em Python-Django. Como objetivos específicos, assume-se: i) pesquisar, compilar e escrever sobre PLN e LLM; ii) pesquisar bibliotecas de PLN; iii) pesquisar e testar bibliotecas Python para LLM; iv) pesquisar, compilar e escrever sobre trabalhos relacionados; v) projetar e implementar um módulo de geração de relatórios via *prompt*; vi) definir e aplicar um ambiente de avaliação e teste.

II. Referencial Teórico

Nesta seção, abordam-se os tópicos centrais relacionados à proposta, com ênfase nas estratégias utilizadas na construção do projeto. Nas seções, são exploradas áreas fundamentais, como a Inteligência Artificial Generativa, sua implementação e integração em sistemas Web.

A. Inteligência Artificial: Processamento da Língua Natural e Aprendizado de Máquina

Segundo Peter Norvig e Stuart Russell [2], a IA é uma área da Ciência da Computação que aborda a capacidade das máquinas de executar tarefas que tenha alguma habilidade ou característica do ser humano, criando um conjunto de vantagens como precisão, rapidez e capacidade de gerenciar grande volumes de memória. Nas diversas áreas da IA, existe uma que foca no reconhecimento e na interpretação da língua humana, comumente chamada de PLN [3], o qual possibilita a comunicação entre o ser humano e a máquina, que faz interpretação do que foi falado ou escrito. Sendo utilizado em várias áreas da IA como o aprendizado de máquina e reconhecimento de padrões [4]. O aprendizado de máquina (do inglês, *Machine Learning* - ML) é outra subárea da IA, que foca na construção de sistemas que aprendem com base em uma fonte de dados já existente [3]. Segundo

Stryker e Holdsworth [5], o estudo de como interpretar as diversas formas de como um humano pode escrever (tratar a língua escrita, por exemplo) foi fundamental para o avanço da geração de conteúdo e a popularização dos assistentes virtuais e *chatbots*. Dessa forma, a IA Generativa (do inglês, *Generative AI* - IA Generativa) é uma subcategoria da Inteligência Artificial que combina aprendizado de máquina e um vasto conjunto de dados para produzir novos conteúdos, fundamentando-se no que já existe [6]. Essas aplicações utilizam o PLN como parâmetro, permitindo que, ao ser processada, retornem uma resposta baseada no formato ou na escrita. Essa resposta pode manifestar-se como um texto, uma imagem ou um som [6].

1) Large Language Models: Todas as aplicações que utilizam IA Generativa fundamentam-se em modelos de linguagem, que operam em conjunto com técnicas de aprendizado de máquina e PLN [7]. Esses modelos são treinados com enormes quantidades de dados, sendo denominados Modelos de Linguagem de Larga Escala (LLM) [8]. Registra-se que há outras fases importantes [9], como: Pré-processamento: antes do treinamento, os dados passam por uma limpeza e transformação; *Tokenização*: o texto é dividido em *tokens*, que podem representar palavras ou partes de palavras, facilitando o processamento e a compreensão do modelo; Filtragem e curadoria: inúmeras vezes, os dados são filtrados para remover conteúdo indesejado ou inadequado, garantindo que o modelo aprenda com informações de qualidade, como se fosse um processo de melhoria ou sintonia (*tuning*).

Com o avanço da tecnologia (como processadores e memórias mais rápidos, sistemas em nuvem, *clusters* computacionais, etc), os *Transformers* originaram LLMs mais sofisticados, como o GPT (*Generative Pre-Trained Transformer*) [10] e o Llama (*Large Language Model Meta AI*) [11]. Esses modelos, desenvolvidos para compreender e gerar texto de maneira mais eficiente, popularizaram o uso da Inteligência Artificial em diversas aplicações, como assistentes virtuais, geração de conteúdo e tradução automática [12]. Segundo a definição sobre os fundamentos da IA Generativa, no Microsoft Learn [7], os modelos *Transformers*, ou Modelos Baseados em Atenção, são arquiteturas treinadas com grandes volumes de texto que permitem uma representação semântica das palavras (ou seja, seus significados e sentidos). Essa abordagem utiliza as relações entre essas palavras para determinar sequências probabilísticas de texto que fazem sentido. A integração dos *Transformers* está presente na maioria das LLMs e serve como uma forma evolutiva para o entendimento e a geração de novos conteúdos em linguagem natural [9]. Esses modelos podem ser facilmente implementados em projetos com o auxílio de ferramentas prontas, como as discutidas na próxima seção.

2) Ferramentas para IA Generativa: Para o desenvolvimento integrado com IA Generativa, a linguagem de programação orientada a objetos Python [13] tem se destacado, fornecendo recursos mais variados, desde bibliotecas até API. Dentro desse ecossistema, o principal *framework* para comunicação é o LangChain [14], que possibilita o desenvolvimento de aplicações utilizando LLMs, oferecendo ferramentas e abstrações que aprimoram a personalização, precisão e relevância das informações geradas pelos modelos. Além disso, essa ferramenta permite que LLMs acessem diferentes conjuntos de dados sem a necessidade de retreinamento [15].

Por sua vez, o Ollama [16] é uma ferramenta que simplifica o processo de implementação de instâncias de LLMs, executando localmente. Sua principal funcionalidade é baixar e configurar um servidor com o modelo Llama, na versão 3.1, mais específico que possui 8 bilhões de parâmetros (8B) da Meta [11]. O Ollama Python [17], é uma biblioteca que permite a integração de modelos de linguagem instanciados no Ollama de maneira simples e eficiente, facilitando a interação com o modelo para requisição e resposta de suas funcionalidades. Com esses recursos, é possível criar um sistema básico de requisição de informações. No entanto, a geração de conteúdo pode ser impactada por grandes volumes de dados, exigindo uma técnica para garantir maior consistência, denominada RAG.

Na geração de conteúdo, quando muitas informações são transmitidas ou solicitadas, a natureza da tecnologia utilizada nas LLMs pode resultar em respostas imprevisíveis [18]. Isso gera problemas como informações falsas, desatualizadas ou fora de contexto, além de respostas baseadas em fontes inventadas e imprecisões gerais de conteúdos com terminologia ambígua ou semelhante. A Geração Aumentada de Recuperação (do inglês, *Retrieval-Augmented Generation*) é uma técnica empregada para aumentar a precisão e a confiabilidade das respostas da IA Generativa, utilizando informações de fontes externas [19]. Esta ferramenta opera por meio da interação direta do modelo de linguagem com um sistema de armazenamento de dados persistente, utilizando consultas para acessar e recuperar informações relevantes [19].

Modelos pré-treinados, desenvolvidos a partir de grandes volumes de dados por empresas, tornam o uso acessível ao usuário comum e abrem novas possibilidades para o desenvolvimento de aplicações de uso geral. A utilização de uma LLM local possibilita seu funcionamento *offline*, evitando a coleta de informações pelas empresas que a hospeda [20]. Um dos principais fatores que limitam a utilização abrangente das LLMs locais é a infraestrutura computacional. Segundo o site *Llama AI Model* [21], os requisitos mínimos de hardware para a operação do modelo incluem um processador de 8 núcleos e pelo menos 16 GB de memória RAM.

B. Remote Procedure Call

No contexto da computação distribuída, uma Chamada de Procedimento Remoto (do inglês, *Remote Procedure Call* - RPC) ocorre quando um programa de computador invoca

uma sub-rotina ou procedimento para ser executado em um espaço de memória distinto, de forma sincronizada. Geralmente, isso envolve outro computador ou uma rede compartilhada de computadores [22]. Essa interação entre máquinas caracteriza uma arquitetura cliente-servidor, na qual o cliente é quem solicita a execução de um procedimento em outro sistema, atuando este como servidor. Utilizando a biblioteca *xmlrpc*, é possível implementar o RPC em Python de maneira simples. A teoria RPC foi abordada devido à inviabilidade de executar uma IA Generativa baseada em LLM em servidores Web tradicionais. Conforme discutido na seção sobre IA Generativa, um dos desafios de executar uma LLM local é a alta demanda computacional, que pode causar instabilidade e alta latência nas respostas do servidor [23]. Uma solução para reduzir essa carga é contratar um serviço especializado, com máquinas externas que atendem ou superam os requisitos necessários para uma implementação mais eficaz. Empresas oferecem esse tipo de serviço, conhecido como computação em nuvem.

Segundo Kolbe [24], computação em nuvem refere-se ao fornecimento de serviços de computação pela internet. É denominada "nuvem" por não exigir a presença de uma máquina física para o funcionamento do serviço, transmitindo a ideia de algo externo, facilmente acessível e abrangente. A computação em nuvem destaca-se pela fácil escalabilidade e pelo custo-benefício de sua utilização. O processamento das informações das LLMs pode ser realizado em um modelo de "pague conforme o uso" (do inglês, *pay-as-you-go*), que oferece uma solução mais econômica e confiável do que uma máquina dedicada [25].

C. Trabalhos correlatos

Os trabalhos correlatos, nesta seção, têm como objetivo contextualizar as pesquisas e tecnologias relevantes para este estudo. O propósito é identificar as possibilidades e desafios que fundamentam o presente trabalho.

O trabalho relacionado de Maurer [26] visa o desenvolvimento de uma interação humano-computador com linguagem natural sem depender de conexão com a Internet, utilizando LLMs locais para a geração de texto ou voz. Além disso, o autor detalha de forma aprofundada e clara os fundamentos da IA Generativa e como ocorre sua integração em um ecossistema Python. No trabalho de Barros [27], a principal relação com esta pesquisa reside na metodologia proposta, para a avaliação das respostas geradas por LLMs na geração de código nas linguagens Python e Lua. A partir de testes, são apresentadas as métricas de avaliação, como precisão, legibilidade e desempenho. O trabalho de Müller [1] tem como principal relevância o sistema desenvolvido, que propõe um serviço capaz de gerenciar o processo de submissão, avaliação e acompanhamento de projetos junto à Comissão Científica (COMIC).

Os estudos como os de Müller [1] e Barros [27] apresentam sistemas relevantes que podem ser aplicados diretamente ao contexto desta pesquisa. Por outro lado, o trabalho de Maurer [26] fornece *insights* sobre como esse módulo pode ser implementado de maneira eficiente e correta.

III. Metodologia

Para a elaboração do texto, utilizou-se pesquisa de revisão bibliográfica. Já para o desenvolvimento deste projeto, optou-se por uma abordagem de metodologia ágil Scrum, com apoio do método Kanban.

As ferramentas utilizadas foram: Linguagem Python; Bibliotecas Python: langchain, ollama, xmlrpc; Modelo linguagem pré-treinado Llama 3.1 8B; Framework LangChain para implementação do RAG. Os encontros para a construção do trabalho foram semanais, utilizando um editor de texto colaborativo e repositório de versionamento de código. O orientador atuou como *Scrum Master*, o aluno como desenvolvedor (*Development Team*), e gestora e administradora do SISGEP-COMIC como cliente e *Product Owner*. A modelagem do sistema concentra-se nos aspectos funcionais, sendo fundamentada pelo *Product Backlog* e os diagramas UML.

Para representar de forma clara a interação entre os dois servidores, foi elaborado um diagrama de classes, apresentado na Figura 1, com o objetivo de estruturar e explicar as etapas envolvidas na geração do relatório.

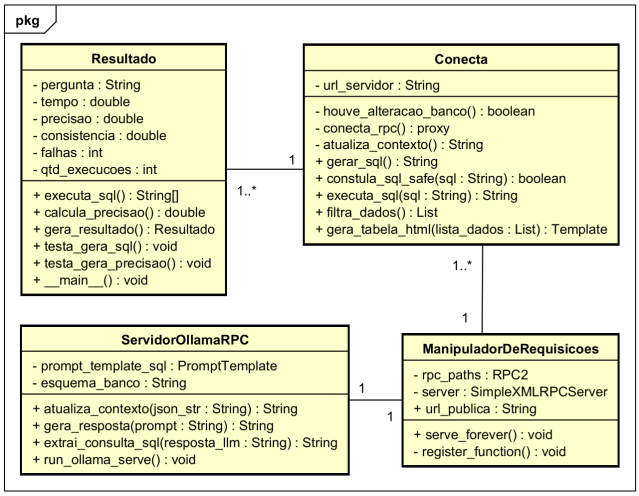

Figura 1. Diagrama de classes do SISGEP-COMIC e Servidor Ollama.

No diagrama, a classe *ServidorOllamaRPC* centraliza a lógica de manipulação do LLM, sendo responsável por gerar respostas e atualizar o contexto do RAG. Além disso, a classe possuí um método para realizar a triagem da resposta gerada para determinar se existe uma instrução SQL usando expressões regulares. Para expor esses métodos via RPC, é criada a classe *ManipuladorDeRequisicoes*, que permite o registro de funções e a conexão a partir em uma URL pública. A classe *Conecta* é responsável por estabelecer a conexão com o servidor Ollama, com os métodos de

geração de consultas e atualização de contexto executados remotamente via RPC. Os demais métodos são executados localmente, incluindo a verificação de segurança da consulta antes da execução no banco de dados. Para a exibição das informações, dois métodos adicionais são utilizados para filtrar e estruturar os dados em uma tabela HTML. A classe *Resultado* será tratada em mais detalhes no estudo de caso, no qual é descrito sua finalidade e principais atributos.

O diagrama de atividade apresentado na Figura 2, ilustra o fluxo de interação entre os dois servidores, detalhando o processo de geração da consulta SQL por meio do *prompt* usando PLN. Já o diagrama de componentes, apresentado na Figura 3, representa esta estrutura, destacando a interação entre os dois servidores, onde um utiliza o *framework* Python-Django do SISGEP-COMIC, e o outro representa o servidor Ollama com a LLM comunicando via RPC.

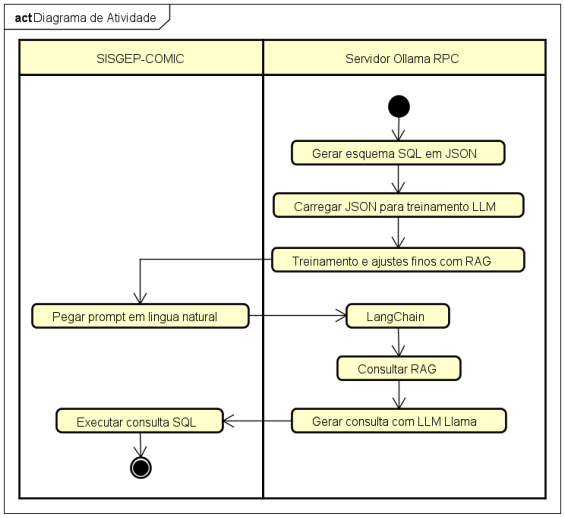

Figura 2. Diagrama de atividade do SISGEP-COMIC e Ollama.

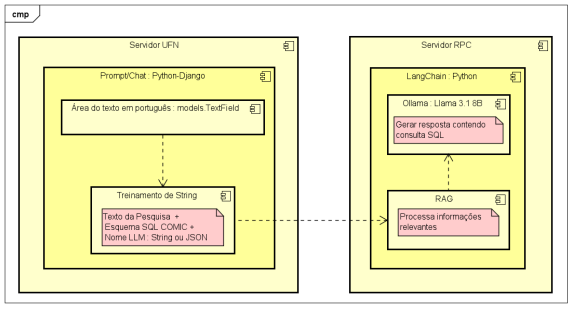

Figura 3. Diagrama de componentes com os dois servidores.

A. Estudo de caso: processos de integração e de avaliação

Para processo de integração, a geração de conteúdo usando IA Generativa é obtida através da ferramenta Ollama para instanciar LLMs já treinadas, no qual já possuem as tecnologias de PLN e ML necessárias para interpretar requisições via *prompt*. O processamento das requisições do cliente é realizado via métodos, executados no servidor via RPC, que é gerenciada pelo *framework* LangChain. A utilização de contextos na base de conhecimentos do módulo RAG é aplicada para gerar consultas de forma mais confiável, tendo como principal base um arquivo JSON contendo o esquema do banco de dados do SISGEP-COMIC. Por fim, a resposta é enviada ao servidor Web de forma textual, contendo a consulta a ser executada. Após a inserção do *prompt*, ele é enviado por meio de uma chamada RPC para um servidor externo. Esse servidor utilizará a LLM para gerar uma consulta SQL com base na estrutura treinada pelo RAG. A consulta gerada é então retornada ao sistema cliente, onde é aplicada ao sistema gerenciador do banco de dados por meio do *framework* Python-Django. Quando a consulta for executada, o sistema retornará as informações solicitadas e as exibirá de forma estruturada ao usuário.

Para implementação do servidor responsável por instanciar o Ollama e expor métodos foi realizada por meio de duas abordagens. A primeira utilizou uma máquina virtual em nuvem, por meio da plataforma *Google Colab*, com acesso a uma *GPU Nvidia T4*. Já a segunda foi executada de maneira local, em uma máquina com sistema operacional Linux, utilizando uma *GPU* dedicada da *Nvidia*. Ambas as abordagens utilizam o mesmo código, diferenciando-se apenas na forma de conexão. No *Colab*, a conexão é viabilizada por um túnel via serviço *ngrok*, permitindo acesso externo ao servidor Ollama.

Para a avaliação do módulo, uma série de testes foi elaborada para garantir que os dados gerados sejam factuais como forma de resultados. As perguntas de avaliação foram executadas de duas maneiras distintas. A primeira segue o processo tradicional, no qual a consulta é criada manualmente para o SGBD, de forma semelhante ao que já é feito no SISGEP-COMIC. A segunda maneira utiliza o módulo gerador de relatórios, empregando as perguntas como *prompt*.

As métricas de avaliação são: i) tempo de execução; ii) precisão dos dados obtidos; iii) consistência das respostas; iv) taxa de falhas. No primeiro item, o tempo é medido em segundos. Para isso, utiliza-se a biblioteca *time* da linguagem Python, que inicia um cronômetro quando o *prompt* é enviado ao sistema e o interrompe quando a resposta é completamente gerada e exibida ao usuário. No segundo item, a precisão é avaliada com base na correspondência entre os dados retornados pela consulta SQL gerada e os dados de referência presentes no banco. Por exemplo, se todas as informações coincidirem com as de referência, a precisão será de 100%. Essa porcentagem é obtida pela fórmula 1, onde G representa o conjunto de dados retornados pela consulta gerada pelo módulo, e R é o conjunto de dados corretos da consulta de referência.

Para comparar os dois conjuntos, uma função no servidor recebe as informações de forma estruturada, realizando a comparação e determinando a diferença.

$$P = G/R \tag{1}$$

No terceiro item, é representado o percentual de consistência da precisão obtida a partir de várias execuções. Para calcular esse percentual, utiliza-se a fórmula 2, na qual é feita a média das precisões P obtidas sobre o número total n de consultas realizadas.

$$C = \frac{1}{n}\sum_{i=1}^{n} P_i \tag{2}$$

No quarto e último item, é definido o percentual de falhas para as consultas executadas que não foram bem-sucedidas. A fórmula 3, descreve como essa porcentagem será obtida, onde $nFalhas$ representa o número de consultas que falharam, e $nTotal$ é o número total de consultas executadas.

$$F = nFalhas/nTotal \tag{3}$$

Para medir a taxa de falhas, é necessário definir o que caracteriza uma consulta com falha. Segundo Jalote [28], uma falha é um comportamento inesperado no software, ocorrido no meio físico, e pode ser determinada pela sua natureza, duração, extensão e valor. Uma consulta com falha pode ocorrer por diversos motivos, como erros de sintaxe ou lógica, ou ainda problemas na execução ou comunicação com o SGBD.

IV. Resultados e Discussões

Nesta seção, são discutidos os resultados obtidos com o desenvolvimento do módulo proposto, bem como seu desempenho em diferentes situações e com base no modelo proposto no estudo de caso.

A. *Implementação do servidor Ollama*

Na Figura 4, são ilustrados os dois principais métodos de chamada remota ao servidor Ollama. O primeiro é responsável por gerar consultas SQL a partir de perguntas em linguagem natural, recebidas por parâmetro. O segundo método tem como finalidade atualizar o contexto do mecanismo do RAG, utilizando como parâmetro dados estruturados no formato JSON.

A geração das respostas pelo modelo de linguagem é feita através do *PromptTemplate* da biblioteca LangChain. A partir desta ferramenta, foi possível criar uma forma de molde dinâmico contendo duas variáveis essenciais, a pergunta do usuário e o esquema do banco de dados. Este *template*, ilustrado na Figura 5, é utilizado sempre que o modelo precisa gerar uma resposta, garantindo que o *prompt* seja processado de maneira estruturada e contextualizada.

Para que esses métodos possam ser acessados remotamente, foi implementada uma nova classe chamada *ManipuladorDeRequisicoes*. Representada na Figura 6, a variável

```python
# (RPC) Define a função que atualiza o contexto usando JSON
def atualizar_contexto(json_str):
    global esquema_banco
    try:
        print('------ ATUALIZANDO CONTEXTO ------')
        if isinstance(json_str, dict):
            esquema_banco = json_str
        else:
            esquema_banco = json.loads(json_str)
        return 'Contexto atualizado com sucesso!'
    except Exception as e:
        print(f"Erro ao atualizar o contexto: {str(e)}")
        return f"Erro ao atualizar o contexto: {str(e)}"
    finally:
        print('CONTEXTO ATUAL > ',esquema_banco)
        print('---- FIM ATUALIZAÇÃO CONTEXTO ----')

# (RPC) Define a função que gera consultas SQL
def gerar_resposta(prompt_usuario: str) -> str:
    print('------ NOVA REQUISIÇÃO DO USUÁRIO ------')
    print('PERGUNTA > ', prompt_usuario)
    try:
        # Gerar a resposta
        llm_response = sql_chain.invoke({
            "input_text"  : prompt_usuario,
            "schema_info" : json.dumps(esquema_banco)
        })
        print('RESPOSTA > ', llm_response)

        # Extrair o SQL usando expressões regulares
        sql_query = extrair_consulta_sql(llm_response)
        print('QUERYSQL > ', sql_query)

        return sql_query
    except Exception as e:
        error_msg = f"Erro ao gerar resposta: {str(e)}"
        print(error_msg)
        return error_msg
    finally:
        print('------- FIM REQUISIÇÃO DO USUÁRIO ------')
```

Figura 4. Código dos métodos locais para serem registrados.

```python
# Define o template (personalidade) que será utilizada pelo modelo.
prompt_template_sql = PromptTemplate.from_template(
    """
    Dado o seguinte esquema de banco de dados SQL:
    {schema_info}

    Converta a seguinte consulta em linguagem natural em uma instrução
    SQL SELECT utilizando apenas o esquema como referência:
    {input_text}

    Regras:
    1. Se perguntando algo não relacionado a consultas,
       responda: **"Eu não sei."**
    2. Ignore tabelas ou colunas não reconhecidas,
       não invente novas.
    3. Responda apenas com consultas SQL entre três crases (```)
       sem explicações ou comentários.
    4. Use os nomes completos das tabelas conforme definidos pelo
       Django (por exemplo, `usuario` torna-se `usuario_usuario`).
    5. Nomes de tabelas e colunas podem conter acentos,
       remova os acentos ao comparar com o texto da consulta.
    6. Para consultas sobre pesquisadores, investigadores
       ou professores, use a tabela `usuario_usuario`.
    7. Considere o texto sem distinção entre maiúsculas
       e minúsculas (`professor` = `PROFESSOR`).
    8. Para perguntas sobre conteúdo ou área de pesquisa, consulte
       a tabela `submissao_submissao` utilizando os campos relevantes.
    """
)
```

Figura 5. Código do servidor Ollama definindo o *PromptTemplate*.

server é um objeto dessa classe, que registra estes dois métodos mencionados anteriormente. Após o registro, o servidor Ollama entra em execução e a *URL* pública de conexão é fornecida ao usuário, possibilitando a integração com outros serviços, como o SISGEP-COMIC.

```
120 def run_ollama_serve():
121   subprocess.Popen(["ollama", "serve"])
122 thread = threading.Thread(target=run_ollama_serve)
123 thread.start()
124 time.sleep(5)
125 # Incializa o modelo usando recém criado llama3.1.
126 llm = OllamaLLM(
127     model="llama3.1",
128 )
129 # Inicializa a chain para fazer chamadas para LLM usando template.
130 sql_chain = prompt_template_sql | llm
131 # Cria o servidor RPC para responder requisições.
132 class ManipuladorDeRequisicoes(SimpleXMLRPCRequestHandler):
133     rpc_paths = ("/RPC2",)
134
135 server = SimpleXMLRPCServer(
136     ("0.0.0.0", 1346),
137     requestHandler=ManipuladorDeRequisicoes,
138     allow_none=True
139 )
140 # Registra as funções para gerar SQL e atualizar contexto via RPC.
141 server.register_function(atualizar_contexto, "atualizar_contexto")
142 server.register_function(gerar_resposta, "gerar_resposta")
143 # Cria URL pública pelo ngrok para acessar fora do Google Colab.
144 try:
145     url_publica = ngrok.connect(1346, bind_tls=True).public_url
146     print("URL Pública: ", url_publica)
147 except Exception as e:
148     print(f"Falha ao conectar com ngrok: {str(e)}")
149     raise
150 # Instancia o servidor e executa sem interrupção
151 print("Servidor XML-RPC em execução...")
152 server.serve_forever()
```

Figura 6. Código da classe *ManipuladorDeRequisicoes* e inicialização.

A implementação local do servidor Ollama mostrou-se significativamente mais complexa do que a alternativa de terceirização por serviços como o *ChatGPT* da *OpenAI*. Essa diferença vem de diversos fatores, como a maior complexidade técnica e os custos associados ao hardware. Além disso, outras limitações também impactaram negativamente os resultados obtidos em relação a serviço de terceiros, conforme será detalhado na subseção da tabela de resultados.

Por outro lado, este método também apresenta vantagens importantes, como o maior controle sobre a manipulação do modelo e a maior independência, uma vez que o sistema pode ser utilizado de forma totalmente *offline* e dentro da própria infraestrutura em que foi implementado.

B. Adaptações no SISGEP-COMIC

Após a implementação do servidor Ollama capaz de responder com consultas SQL, a próxima etapa foi implementar a interação com o usuário em uma aplicação Web usando *framework* Django, neste caso, o SISGEP-COMIC.

A Figura 7 apresenta três métodos da classe *Conecta*, utilizados para a conexão com o servidor Ollama por meio de uma URL pública. A própria classe também é responsável por estabelecer essa conexão, invocando o método correspondente sempre que for necessário gerar uma instrução SQL ou atualizar o contexto.

```
40     @staticmethod
41     def conecta_rpc():
42         url_servidor = config('GERADORSQL_URL')
43         try:
44             # Verifica se houve alteração pelo migrations
45             if Conecta.houve_alteracao_banco():
46                 Conecta.atualiza_contexto()
47
48             proxy = xmlrpc.client.ServerProxy(url_servidor)
49             return proxy
50         except Exception as e:
51             print('Erro ao conectar no servidor: ', e)
52             return 'Não foi possível conectar no servidor. Tente novamente mais tarde.'
53     @staticmethod
54     def atualiza_contexto():
55         try:
56             proxy = Utils.gera_esquema_banco()
57             with open("esquema_banco.json", "r") as file:
58                 #Lê o arquivo Json e envia para o servidor
59                 json_data = file.read()
60                 resposta = proxy.atualiza_contexto(json_data)
61                 return resposta
62         except Exception as e:
63             return f"Erro ao atualizar contexto. Exceção: {str(e)}"
64     @staticmethod
65     def gera_sql(pergunta):
66         try:
67             proxy = Conecta.conecta_rpc()
68             resposta = proxy.gera_resposta(pergunta)
69             return resposta
70         except Exception as e:
71             print('Erro ao gerar SQL: ', e)
72             return 'Não foi possível conectar no servidor. Tente novamente mais tarde.'
```

Figura 7. Códigos utilizando RPC da classe *Conecta*.

Para recuperar as informações, a Figura 8 apresenta o método responsável por realizar a conexão com o banco de dados, utilizando a ferramenta *cursor* para executar as instruções SQL. A consulta ocorre apenas se a verificação de segurança for bem-sucedida, ou seja, se o método de validação retornar verdadeiro. Ao final do processo, o objeto *cursor* e a conexão com o banco de dados são fechados.

```
98     @staticmethod
99     def executa_sql(sql):
100        try:
101            # Verifica se a consulta é segura
102            if not Conecta.checa_consulta_segura(sql):
103                return "Uma consulta potencialmente insegura foi detectada. Por favor, tente novamente."
104            # Conecta no banco com cursor e obtém os resultados
105            with connection.cursor() as cursor:
106                cursor.execute(sql)
107                resultados = cursor.fetchall()
108                # Se não houver resultados, retorna a mensagem
109                if not resultados:
110                    resultados = "Nenhum resultado relevante foi encontrado."
111                # Obtém os nomes dos campos
112                nomes_campos = [i[0].upper() for i in cursor.description]
113                cursor.close()
114            # Se não houver "order by" na consulta, ordena a lista de listas
115            if "order by" not in sql.upper():
116                lista_dados = sorted(resultados, key=lambda x: x[0])
117            else:
118                lista_dados = resultados
119            # Filtra os resultados
120            lista_filtrada = Conecta.filtra_resultados(lista_dados, nomes_campos)
121            # Gera a tabela HTML com os resultados filtrados
122            return Conecta.gera_tabela_html(nomes_campos, lista_filtrada)
123        except Exception as e:
124            print('Erro ao executar a consulta: ', e)
125            return f"Erro na execução da consulta. Tente gerar a consulta novamente."
```

Figura 8. Código do método para executar SQL da classe *Conecta*.

Na etapa de exibição dos resultados, o código da Figura 8 chama no final, outros dois métodos que filtram os nomes dos campos e os dados de cada linha em listas. Em seguida, essas informações são estruturadas em uma tabela HTML usando a ferramenta *Template* do Django.

Na Figura 9, é apresentado o código do método responsável por gerar a tabela HTML junto de sua localização na

camada *view* do relatório. Como o objeto resposta é de um tipo reconhecido pelo Django, a renderização em HTML dos dados ocorre automaticamente, porém apenas quando é utilizada a palavra-chave *safe*, junto da pergunta e da consulta executada.

```python
conecta_llm.py
    @staticmethod
    def gera_tabela_html(nomes_campos, lista_dados):
        template_str = """
                        <h2 style="text-align:center;">Tabela de Resposta</h2>
                        <table class="table table-hover">
                            <thead>
                                <tr>
                                    {% for campo in nomes_campos %}
                                        <th>{{ campo }}</th>
                                    {% endfor %}
                                </tr>
                            </thead>
                            <tbody>
                                {% for item in lista_dados %}
                                    <tr>
                                        {% for valor in item %}
                                            <td>{{ valor }}</td>
                                        {% endfor %}
                                    <tr>
                                {% endfor %}
                            </tbody>
                        </table>
                        <p><b>Total de registros:</b> {{ lista_dados|length }}</p>
                        """
        template = Template(template_str)
        context = Context({
            'nomes_campos': nomes_campos,
            'lista_dados': lista_dados
        })
        return template.render(context)
```

```html
relatorio_detail.html
    <div class="panel panel-default">
        <div class="panel-heading">
            Respostas do processo
        </div>
        <div class="panel-body">
            {% if object.resposta %}
                {{ object.resposta|safe }}
            {% else %}
                <span class="label label-warning" >
                    Resposta não disponível
                </span>
            {% endif %}
            <br>
```

Figura 9. Código do método para gerar uma tabela HTML.

Como resultado, a visão do usuário, representada na Figura 10, mostra o que é retornado para a seguinte pergunta: Informe o nome, o e-mail e o currículo Lattes de todos os usuários cadastrados que são professores.

No servidor Ollama, a Figura 11 representa a visão do terminal durante a execução dessa pergunta. Nela é possível observar a requisição do usuário, que ao terminar de processar, retorna como resposta a consulta SQL.

A partir dos métodos apresentados, foi possível utilizar componentes de ambos os sistemas para aplicar a sequência de testes proposta no estudo de caso, organizando-os em uma tabela de resultados.

C. Elaboração da tabela de resultados

As métricas de avaliação propostas no estudo de caso fornecem uma base quantitativa essencial medir a usabilidade do produto desenvolvido. Permitindo através de sua análise determinar se o módulo, em seu estado atual, está apto para uso em produção. Para obter essas métricas, foi implementado um código isolado do *framework* Django,

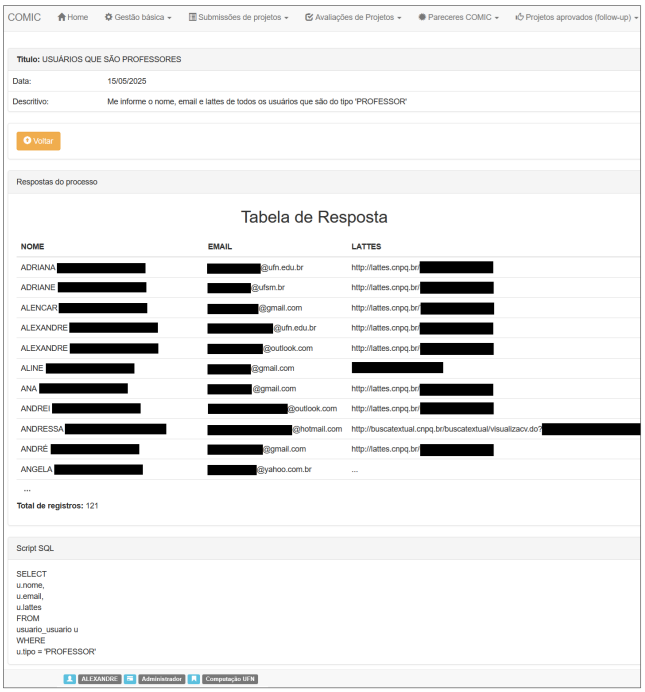

Figura 10. Resultado da pergunta de exemplo.

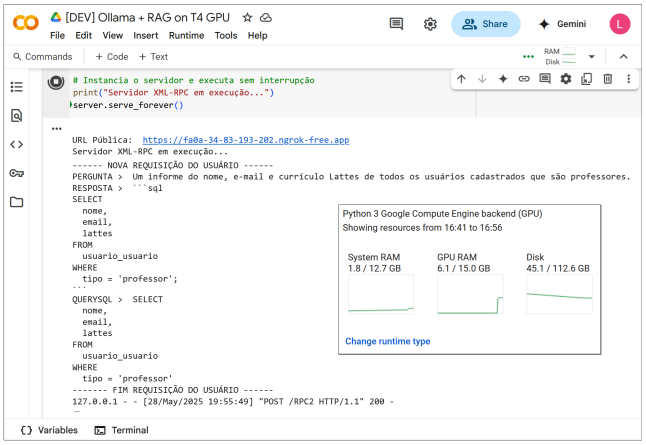

Figura 11. Tela do servidor Ollama executando no ambiente *Google Colab*.

além de uma nova classe específica para agrupar os dados, seguindo as boas práticas da programação orientada a objetos. A implementação desse sistema faz uso da classe *Resultado*, representada no diagrama de classes da Figura 1.

Conforme detalhado no diagrama, as métricas foram geradas por meio de diversos métodos. O principal deles, representado na Figura 12, é responsável por gerar e retornar uma lista de objetos *Resultado*. Esse método mede o tempo para gerar o SQL de cada pergunta, além de chamar outro método para calcular a precisão, contabilizar as falhas por meio do tratamento de exceções e avaliar a consistência dos dados ao realizar múltiplas iterações da mesma pergunta.

Como parâmetros, ele recebe uma lista de perguntas, uma lista de comandos SQL de referência e a quantidade de consultas realizada para cada pergunta.

```python
def gera_resultados(lista_perguntas, lista_sql_referencia, qtd_execucoes):
    lista_resultados = []
    if(len(lista_perguntas) != len(lista_sql_referencia)):
        raise ValueError("A lista de perguntas e SQL de ref. não tem o mesmo tamanho!")
    for pergunta, sql_referencia in zip(lista_perguntas, lista_sql_referencia):
        lista_tempos = []
        lista_precisao = []
        consistencia = 0.0
        qtd_falhas = 0
        for i in range(qtd_execucoes):
            try:
                #1. Gera a consulta SQL via RPC (Temporizado)
                tempo_inicial = time.time()
                sql_gerado = Conecta.gera_sql(pergunta)
                print(f"[{i+1}] SQL : {sql_gerado}")
                tempo_passado = time.time() - tempo_inicial
                lista_tempos.append(round(tempo_passado, 2))
                #2. Executa a consulta SQL gerada e de referência
                (linha_ref, campos_ref) = executa_sql(sql_referencia)
                (linha_ger, campos_res) = executa_sql(sql_gerado)
                #3. Filtra os resultados
                lista_referencia = Conecta.filtra_resultados(linha_ref, campos_ref)
                lista_gerada = Conecta.filtra_resultados(linha_ger, campos_res)
                #4. Calcula a precisão
                precisao = calcular_precisao(lista_referencia, lista_gerada)
                lista_precisao.append(precisao)
            except Exception as e:
                # Se ocorrer um erro, contabiliza a falha e continua
                print(f"Erro na execução {i}: {e}")
                qtd_falhas += 1
                lista_precisao.append(0.0)
        #5. Calcula a consistência
        consistencia = sum(lista_precisao) / qtd_execucoes
        #6. Cria o objeto Resultado e adiciona à lista
        resultado = Resultado(pergunta, lista_tempos, lista_precisao,
                              consistencia, qtd_falhas, qtd_execucoes)
        lista_resultados.append(resultado)
    return lista_resultados
```

Figura 12. Método utilizado para gerar resultado e suas métricas.

Para fins de teste e validação dos métodos apresentados, uma lista de duas perguntas simples foi elaborada. A Figura 13 mostra essas perguntas e seu respectivo SQL de referência, as métricas do resultado dessas perguntas pode ser visualizada na saída do terminal.

```python
lista_perguntas = [
 "Me informe o nome, email e data de nascimento dos usuários do tipo administrador",
 "Me informe o nome, sigla e site de cada instituição"]

lista_sql_ref = [
 "SELECT nome, email, data_nasc FROM usuario_usuario WHERE tipo = 'ADMINISTRADOR'",
 "SELECT nome, sigla, site FROM instituicao_instituicao"]

# Cria a lista de resultados com 5 ocorrências para cada pergunta na lista.
lista_resultados = gera_resultados(lista_perguntas, lista_sql_ref, 5)
print(*lista_resultados, sep='\n')
```

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS TAILSCALE

```
--------------------------------------------------
Pergunta: Me informe o nome, email e data de nascimento dos usuários do tipo administrador
Tempo: [10.31, 3.5, 3.83, 2.75, 3.56]
Tempo médio: 4.79 s
Precisão: [1.0, 1.0, 1.0, 1.0, 1.0]
Precisão média: 1.00
Consistência: 100.00%
Falhas: 0
Quantidade de execuções: 5
--------------------------------------------------
--------------------------------------------------
Pergunta: Me informe o nome, sigla e site de cada instituição
Tempo: [3.04, 2.91, 3.06, 2.93, 2.92]
Tempo médio: 2.97 s
Precisão: [1.0, 1.0, 1.0, 1.0, 1.0]
Precisão média: 1.00
Consistência: 100.00%
Falhas: 0
Quantidade de execuções: 5
--------------------------------------------------
```

Figura 13. Exemplo para duas perguntas e geração do resultado.

Após confirmar que todos os métodos estavam funcionando corretamente, foi possível realizar o teste final com as dez perguntas propostas no estudo de caso. Sendo elas:

1) Informe o nome, curso e área de todos os usuários.
2) Informe o nome, e-mail e Lattes de todos os usuários que são do tipo professor.
3) Informe o nome, e-mail e último acesso de todos os usuários do tipo administrador.
4) Informe o nome, sigla e site de cada instituição.
5) Informe o local de execução, área e curso de cada submissão cadastrada.
6) Informe quantos usuários cadastrados nasceram antes do ano 2000.
7) Informe o nome, área de conhecimento e curso pós de todos os usuários que tem curso de pós graduação.
8) Informe o nome do avaliador suplente e o seu parecer de suplente para todas as avaliações.
9) Informe o a descrição do edital, o nome, e-mail e Lattes do responsável e título de todas as submissões realizadas no edital de numero 2021.
10) Informe o nome do avaliador, a data de avaliação do responsável e o parecer do avaliador suplente para todas as avaliações que foram realizadas pelo avaliador responsável Cristina De Freitas Rodrigues.

Para calcular a precisão e consistência dessas perguntas, a Figura 14 representa as instruções SQL de referência de cada pergunta, ordenadas por nível de complexidade para simular um cenário de melhor caso e pior caso.

```sql
lista_sql_ref_final = [
  SELECT nome, curso_graduacao_vinculado, area_conhecimento_cnpq
  FROM usuario_usuario,

  SELECT nome, email, lattes
  FROM usuario_usuario
  WHERE tipo = 'PROFESSOR',

  SELECT nome, email, last_login
  FROM usuario_usuario
  WHERE tipo = 'ADMINISTRADOR',

  SELECT nome, sigla, site
  FROM instituicao_instituicao,

  SELECT local_execucao, area, curso_graduacao_vinculado
  FROM submissao_submissao,

  SELECT COUNT(*) AS total_usuarios
  FROM usuario_usuario
  WHERE data_nasc < '2000-01-01',

  SELECT nome, area_conhecimento_cnpq, curso_pos_graduacao
  FROM usuario_usuario
  WHERE curso_pos_graduacao IS NOT NULL,

  SELECT u.nome AS avaliador_suplente, a.parecer_avaliador_suplente
  FROM avaliacao_avaliacao a
  LEFT JOIN usuario_usuario u ON a.avaliador_suplente = u.id,

  SELECT e.descricao, u.nome, u.email, u.lattes, s.titulo
  FROM submissao_submissao s
  JOIN edital_edital e ON s.edital = e.id
  JOIN usuario_usuario u ON s.responsavel = u.id
  WHERE e.numero = '2021',

  SELECT u.nome AS avaliador_responsavel, a.dt_avaliacao_responsavel,
  FROM avaliacao_avaliacao a              a.parecer_avaliador_suplente
  JOIN usuario_usuario u ON a.avaliador_responsavel = u.id
  WHERE u.nome = 'CRISTINA DE FREITAS RODRIGUES'
]
```

Figura 14. Lista de SQL de referência para testar cada pergunta.

Com a execução das dez perguntas, cada uma com dez iterações adicionais para medir a consistência. Foi possível extrair as informações utilizando o tempo médio e a precisão média como generalização dos resultados obtidos. A Tabela I representa esses dados, sendo possível chegar a uma conclusão sobre a qualidade do sistema e se está pronto para ser utilizado em produção.

Tabela I
TABELA DE DESEMPENHO POR PERGUNTA

Pergunta	Tempo	Precisão	Consistência	Falhas	Consultas
01	3,20s	100%	100%	0%	10
02	3,34s	100%	100%	0%	10
03	3,34s	100%	100%	0%	10
04	2,86s	100%	100%	0%	10
05	4,09s	0%	0%	100%	10
06	2,86s	100%	100%	0%	10
07	5,58s	100%	100%	0%	10
08	3,58s	0%	0%	100%	10
09	3,26s	0%	0%	100%	10
10	3,79s	0%	0%	100%	10

A partir da análise dos resultados obtidos e da realização de diversos testes adicionais no sistema SISGEP-COMIC, concluiu-se que o sistema é capaz de gerar consultas SQL, embora dependa de diversos fatores que dificultam a generalização do módulo. Ao testar as mesmas perguntas, mas sem a ajuda de ambiguidades específicas do SISGEP-COMIC nas regras do *PromptTemplate* (linha 99 da Figura 5) , observou-se uma redução significativa na precisão e consistência das respostas. Concluindo a necessidade de atenção especial à estrutura do contexto do banco de dados utilizado (linha 25 da Figura 8), bem como um ajuste fino nas regras e diretrizes definidas no *PromptTemplate*.

A partir dos resultados apresentados, foi possível avaliar o desempenho do módulo como satisfatório. A resposta do sistema é precisa para perguntas mais simples e bem definidas, mas tende a perder essa precisão diante de ambiguidades, seja na formulação da pergunta ou na própria estrutura do banco de dados fornecida. Outro fator relevante é o desempenho do modelo adotado. Ao utilizar a LLM *GPT-4o*, disponibilizada pelo serviço da *OpenAI*, obteve-se um aumento significativo na precisão, além de otimizações e boas práticas específicas para cada cenário, resultando em tempos de resposta mais rápidos para cada pergunta testada.

V. CONCLUSÕES

Neste trabalho, foram explorados os fundamentos teóricos da Inteligência Artificial Generativa, com ênfase no processamento de linguagem natural e nos modelos de linguagem. Além disso, foram abordadas técnicas de contextualização com RAG e ferramentas como LangChain e Ollama, que viabilizam a integração de LLMs locais. Os trabalhos correlatos, como os de Müller [1], Maurer [26] e Barros [27], forneceram uma base sólida ao destacarem, respectivamente, a relevância do sistema SISGEP-COMIC, a implementação de LLMs locais e a avaliação de respostas geradas por modelos de linguagem. Esses estudos reforçaram tanto a viabilidade quanto os desafios envolvidos no desenvolvimento de um módulo de interação textual para a geração de relatórios.

A implementação do módulo proposto envolveu a integração de um servidor Ollama, executando o modelo Llama 3.1 8B, configurado para operar localmente ou na nuvem (via *Google Colab*), utilizando chamadas remotas de procedimento (RPC) para gerar consultas SQL no sistema web SISGEP-COMIC. Nesse sistema, a classe *Conecta* foi desenvolvida para gerenciar a interação com o servidor Ollama, validar consultas SQL e exibir os resultados em tabelas HTML. No servidor Ollama, o *framework* LangChain, com suporte ao *PromptTemplate*, permitiu a conversão de *prompts* em linguagem natural em consultas SQL, utilizando o RAG para aumentar a precisão com base no esquema do banco de dados, descrito em JSON.

Os resultados apresentados na Tabela I indicam que o módulo foi capaz de gerar consultas SQL com precisão adequada e consistência para perguntas simples, com tempos de execução entre três e seis segundos, sem ocorrência de falhas. No entanto, perguntas mais complexas apresentaram altas taxas de falhas e ausência de precisão, o que evidencia limitações na interpretação de consultas mais ambíguas.

Com o objetivo de aprimorar os resultados, recomenda-se para trabalhos futuros: i) testar diferentes LLMs locais mais recentes, como *DeepSeek R1* ou versões mais robustas do Llama, visando melhorar a precisão em perguntas mais complexas; ii) aprofundar o treinamento das regras do *PromptTemplate*, ajustando o contexto do RAG para melhor lidar com ambiguidades; iii) avaliar o módulo utilizando assinaturas de serviços de LLMs terceirizados, como *OpenAI GPT-4* ou *xAI Grok-3*, para superar as limitações de hardware do servidor local.

REFERÊNCIAS

[1] P. T. Müller e A. O. Zamberlan. *Módulo Sisgep-Comic Gestão De Avaliação De Projetos*. Santa Maria, RS, Brasil. Disponível em https://tfgonline.lapinf.ufn.edu.br: Trabalho de Conclusão de Curso Sistemas De Informação - Universidade Franciscana (UFN), 2020.

[2] Stuart J Russell e Peter Norvig. *Artificial intelligence: a modern approach*. Pearson, 2016.

[3] Oracle. *What Is Machine Learning?* 2024. URL: https://www.oracle.com/artificial-intelligence/machine-learning/what-is-machine-learning/ (acesso em 30/09/2024).

[4] B. K. Mishra e R. Kumar. *Natural language processing in artificial intelligence*. CRC Press, 2020.

[5] Cole Stryker e Jim Holdsworth. *What is NLP (natural language processing)?* 2024. URL: https://www.ibm.com/topics/natural-language-processing (acesso em 21/09/2024).

[6] Microsoft Learn. *What is generative AI?* 2024. URL: https://learn.microsoft.com/en-us/training/modules/fundamentals-generative-ai/2-what-is-generative-ai (acesso em 20/10/2024).

[7] Microsoft Learn. *What are language models?* 2024. URL: https://learn.microsoft.com/en-us/training/modules/fundamentals-generative-ai/3-language%20models (acesso em 29/10/2024).

[8] Cloudflare. *What is a large language model (LLM)?* 2024. URL: https://www.cloudflare.com/en-gb/learning/ai/what-is-large-language-model/ (acesso em 29/10/2024).

[9] Ashish Vaswani et al. *Attention Is All You Need*. 2017. URL: https://arxiv.org/abs/1706.03762 (acesso em 16/04/2024).

[10] OpenAI. *ChatGPT - Overview*. 2024. URL: https://openai.com/chatgpt/overview/ (acesso em 05/10/2024).

[11] Meta. *Introducing Llama 3.1: Our most capable models to date*. 2024. URL: https://ai.meta.com/blog/meta-llama-3-1/ (acesso em 09/10/2024).

[12] J. D'Souza. *A Review of Transformer Models*. Set. de 2023. DOI: 10.48366/r640001. URL: https://www.researchgate.net/publication/373757234_A_Review_of_Transformer_Models (acesso em 24/10/2024).

[13] Python. *Welcome to Python.org*. 2024. URL: https://www.python.org/ (acesso em 02/10/2024).

[14] H. Chase. *LangChain*. 2024. URL: https://www.langchain.com/ (acesso em 28/10/2024).

[15] Amazon. *What is LangChain?* 2024. URL: https://aws.amazon.com/what-is/langchain/ (acesso em 31/10/2024).

[16] Ollama. *Ollama - Get up and running with large language models.* 2024. URL: https://ollama.com/ (acesso em 10/10/2024).

[17] Ollama. *Ollama Python Library*. 2024. URL: https://github.com/ollama/ollama-python (acesso em 28/10/2024).

[18] Amazon. *What is Retrieval-Augmented Generation?* 2024. URL: https://aws.amazon.com/what-is/retrieval-augmented-generation/ (acesso em 30/10/2024).

[19] R. Merritt. *What Is Retrieval-Augmented Generation, aka RAG?* 2024. URL: https://blogs.nvidia.com/blog/what-is-retrieval-augmented-generation/ (acesso em 30/10/2024).

[20] OpenAI. *OpenAI - Privacy policy*. 2024. URL: https://openai.com/policies/row-privacy-policy/ (acesso em 22/10/2024).

[21] Llama AI Model. *Llama 3.1 Requirements*. 2024. URL: https://llamaimodel.com/requirements/ (acesso em 05/10/2024).

[22] B. J. Nelson. "Remote Procedure Call". Also CMU-CS-81-119. Tese de dout. Xerox Palo Alto Research Center, mar. de 1981.

[23] C.Z. Xu. *Scalable and Secure Internet Services and Architecture*. Chapman & Hall/CRC Computer and Information Science Series. CRC Press, 2005. ISBN: 9781420035209. URL: https://books.google.com.br/books?id=QnXLBQAAQBAJ.

[24] A. K. Júnior. *Computação em nuvem*. 1ª ed. E-book. São Paulo: Contentus, 2020.

[25] D. McAuley. "AI and LLMs in Cloud Computing: Challenges and Opportunities". Em: *Advances in Computer Sciences* 7.1 (2024).

[26] P. G. G. Maurer e A. O. Zamberlan. *Protótipo de Interação Humano-Computador para Processamento da Língua Natural em LLMs*. Santa Maria, RS, Brasil. Disponível em https://tfgonline.lapinf.ufn.edu.br: Trabalho de Conclusão de Curso Sistemas De Informação - Universidade Franciscana (UFN), 2024.

[27] G. R. C. Barros e M. O. Martins. *Comparative Of Source Code Generated By Principal Llm Generators For Python And Lua Languages*. Santa Maria, RS, Brasil. Disponível em https://tfgonline.lapinf.ufn.edu.br: Trabalho de Conclusão de Curso Ciência Da Computação - Universidade Franciscana (UFN), 2024.

[28] P. Jalote. *Fault Tolerance in Distributed Systems*. PTR Prentice Hall, 1994. ISBN: 9780133013672. URL: https://books.google.com.br/books?id=PZzSngEACAAJ.